

Topic 3: Introduction to Data Preparation

Instructor: Dr. Akash Gupta

Table of contents

1	Introduction to Data Preparation	2
2	Handling Missing Values	3
2.1	Point Value Approaches	3
2.2	K-Nearest Neighbors (KNN)	4
2.3	Multivariate Imputation by Chained Equations (MICE)	5
3	Handling Duplicates	7
4	Data Encoding	8
4.1	One-Hot Encoding	8
4.2	Label Encoding	9
5	Data Transformation	10
5.1	Logarithmic Transformation	10
5.2	Square Root Transformation	11
5.3	Box-Cox Transformation	12
5.4	Data Binning	14
6	Data Scaling	15
6.1	Min-Max Scaling (Normalization)	16
6.2	Z-score (Standardization)	17
6.3	Differences between Normalization and Standardization	19
7	Variable Selection	20
7.1	Forward Elimination	20
7.2	Backward Elimination	20
7.3	Importance of Variable Selection in Modeling	21
8	Feature Engineering	22
8.1	Polynomial Features	23

8.2 Interaction Terms	23
9 Underfitting and Overfitting	24
9.1 Definition	24
9.2 Ridge Regression (L2 Regularization)	26
9.3 Lasso Regression (L1 Regularization)	27
10 Model Evaluation	29
10.1 Accuracy	29
10.2 Precision	31
10.3 Recall	31
10.4 F1-Score	31
10.5 ROC-AUC	32
10.6 K-Fold Cross-Validation	36
11 Bias and Variance	37

1 Introduction to Data Preparation

Data preparation is a crucial step in building predictive models. It includes a series of processes to transform raw data into a clean and usable format, enabling data analyst to apply various analytical techniques and derive insights efficiently.

Importance of Data Preparation in Predictive Analytics

In predictive analytics, the quality of the output is often directly related to the quality of the input data. Data preparation can be thought of as a multi-step process aimed at cleaning and organizing data. The importance can be highlighted through the following aspects:

1. **Improving Data Quality:** Raw data is often messy, containing noise, inconsistencies, and missing values. By addressing these issues, data preparation improves the reliability of results derived from the data.
2. **Enhancing Model Performance:** Machine learning models rely on high-quality data. Properly prepared data allows models to learn patterns effectively.
3. **Facilitating Compliance and Governance:** In many industries, there are stringent regulations on data usage. Data preparation processes often include compliance checks ensuring that data adheres to these regulations.

The data preparation process consists of several stages, each playing a unique role in transforming raw data into a format suitable for analysis. Here are the key stages:

Data Cleaning: - Handling missing values - Removing duplicates - Correcting inconsistencies (e.g., date formats, spelling errors)

Data Transformation: - Normalization or standardization - Encoding categorical variables (e.g., one-hot encoding, dummy variables) - Feature engineering to create new informative features

Data Reduction: Reducing the dataset size for efficiency through techniques like dimensionality reduction or sampling.

2 Handling Missing Values

2.1 Point Value Approaches

- **Imputation** – Replace missing values with estimated values:
 - Use Mean, Median, or Mode for numerical data.
 - Apply Forward/Backward Fill for time series data.
- **Mark Missingness** – Introduce a new feature to indicate whether a value is missing.
- **Remove Rows/Columns** – If only a few values are missing, dropping them may be a simple solution.

Example: For a dataset with two features, if one entry is missing as follows:

Feature 1	Feature 2
1	2
2	3
3	NaN
4	5

```

import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer

# Create DataFrame with missing values
data = {"Feature 1": [1, 2, 3, 4], "Feature 2": [2, 3, np.nan, 5]}
df = pd.DataFrame(data)

# Apply Mean Imputation
imputer = SimpleImputer(strategy="mean") # Using mean to fill missing values
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

# Print before and after imputation
print("Original DataFrame:")
print(df)

print("\nDataFrame after Mean Imputation:")
print(df_imputed)

```

Feature 1	Feature 2
1	2
2	3
3	3.33
4	5

2.2 K-Nearest Neighbors (KNN)

The KNN imputation method uses the k -nearest neighbors to predict the missing value. It computes the distances (often Euclidean) between data points and uses the average (or weighted average) of the k nearest neighbors.

1. Calculate the distance between the point with missing value and all other points.
2. Identify the k closest data points.
3. Impute the missing value using the mean (or median) of these neighbors.

Example: For a dataset with two features, if one entry is missing as follows:

Feature 1	Feature 2
1	2
2	3
3	NaN
4	5

Assuming $k = 2$:

- Distances for the third row (with missing value) to other points:
 - Distance to Row 1: $\sqrt{(3 - 1)^2}$ (use only known values)
 - Distance to Row 2: $\sqrt{(3 - 2)^2}$
 - Distance to Row 4: $\sqrt{(3 - 4)^2}$

The distances yield Rows 1 and 4 being the closest, then the average of Feature 2 from these entries would be used ($\frac{3+5}{2}$).

```
import numpy as np
import pandas as pd
from sklearn.impute import KNNImputer

# Create DataFrame with missing values
data = {"Feature 1": [1, 2, 3, 4], "Feature 2": [2, 3, np.nan, 5]}
df = pd.DataFrame(data)

# Apply KNN Imputer
imputer = KNNImputer(n_neighbors=2) # Using 2 nearest neighbors for imputation
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

# Print before and after imputation
print("Original DataFrame:")
print(df)

print("\nDataFrame after KNN Imputation:")
print(df_imputed)
```

2.3 Multivariate Imputation by Chained Equations (MICE)

MICE models each feature with missing values as a function of other features, iteratively imputing missing data for each feature. This generally involves the following steps:

1. Specify a model for each feature with missing values conditioned on other features.
2. Use the specified models to perform multiple imputation creating several datasets.
3. Pool results for consistent estimates.

Example: Consider features : Age, Income, and Education Level:

- Assume Age has some missing values.
- Predict Age based on Income and Education Level using regression techniques.
- Impute the missing Age iteratively over several passes until convergence.

```
import numpy as np
import pandas as pd
from sklearn.experimental import enable_iterative_imputer # Enable MICE
from sklearn.impute import IterativeImputer
from sklearn.linear_model import LinearRegression

# Create DataFrame with missing Age values
data = {
    "Age": [25, np.nan, 30, 35, np.nan, 40, 45, np.nan, 50, 55],
    "Income": [3000, 3500, 4000, 4500, 5000, 6000, 7000, 7500, 8000, 8500],
    "Education Level": [12, 14, 16, 12, 18, 16, 14, 12, 16, 18] # Years of education
}

df = pd.DataFrame(data)

# Apply MICE (Multiple Imputation by Chained Equations)
imputer = IterativeImputer(estimator=LinearRegression(), max_iter=10, random_state=42)
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

# Print before and after imputation
print("Original DataFrame with Missing Values:")
print(df)

print("\nDataFrame after MICE Imputation:")
print(df_imputed)
```

3 Handling Duplicates

Duplicate entries can skew analysis and lead to incorrect conclusions. It is important to identify and handle these duplicates effectively.

1. **Pandas' duplicated() Function:** This function returns a boolean Series indicating duplicate rows.
2. **Counting Duplicates:** Use `sum()` on the result of the `duplicated()` method to find the total number of duplicates.

Example

Continuing from the student grades dataset, suppose we're collecting multiple submissions.

Student	Math	Science	English
A	90	85	NaN
B	95	75	88
B	95	75	88
C	80	NaN	93
D	85	90	87

```
import pandas as pd
import numpy as np

# Create DataFrame
data = {
    "Student": ["A", "B", "B", "C", "D"],
    "Math": [90, 95, 95, 80, 85],
    "Science": [85, 75, 75, np.nan, 90],
    "English": [np.nan, 88, 88, 93, 87]
}

df = pd.DataFrame(data)

# Find duplicated rows
duplicates = df.duplicated()

# Display the original DataFrame with a column indicating duplicates
df["Is_Duplicate"] = duplicates
```

```
print("Original DataFrame with Duplicates Marked:")
print(df)
```

To remove duplicates:

```
df_cleaned = df.drop_duplicates()
```

4 Data Encoding

4.1 One-Hot Encoding

One-hot encoding transforms categorical variables into a format that can be provided to ML algorithms to improve predictions. This technique creates new binary columns for each category in the data.

For a categorical variable with n unique values, one-hot encoding generates n new binary columns. For instance, if you have a variable `Color` with values `{Red, Blue, Green}`, one-hot encoding will convert it into three binary columns.

For example:

- Original Data:

Color
Red
Blue
Green
Blue

- One-Hot Encoded Data:

	Red	Blue	Green
1	0	0	0
0	1	0	0
0	0	1	0
0	1	0	0


```
import pandas as pd

# Create DataFrame
data = {"Color": ["Red", "Blue", "Green", "Blue"]}
df = pd.DataFrame(data)

# Apply One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=["Color"])

# Display the result
print("Original DataFrame:")
print(df)

print("\nOne-Hot Encoded DataFrame:")
print(df_encoded)
```

Mathematical Representation

If X is the categorical variable and has n categories $C_1, C_2, \dots, C_n$, the one-hot encoding can be mathematically represented as:

$$X_i = \begin{cases} 1 & \text{if } X = C_i \\ 0 & \text{otherwise} \end{cases}$$

4.2 Label Encoding

Label encoding assigns each unique category to a unique integer. Though simpler, this method can introduce an unintended ordinal relationship. For example, assigning labels to the color variable:

- Original Data:

Color
Red
Blue
Green
Blue

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Create DataFrame
data = {"Color": ["Red", "Blue", "Green", "Blue"]}
df = pd.DataFrame(data)

# Apply Label Encoding
encoder = LabelEncoder()
df["Color_Encoded"] = encoder.fit_transform(df["Color"])

# Display the result
print(df)
```

- Label Encoded Data:

Color	Encoded Value
Red	2
Blue	0
Green	1
Blue	0

5 Data Transformation

5.1 Logarithmic Transformation

Logarithmic transformations are particularly useful for reducing skewness in data and compressing the range of data values.

Mathematical Definition: If x is our original data, the logarithmic transformation can be defined as:

$$y = \log(x)$$

where $\log$ can be the natural logarithm (base e), common logarithm (base 10), or any other base.

Example : Consider a dataset of income levels in thousands: ([10, 20, 50, 100, 200, 500]).

1. **Transform Data:**

- Apply the natural logarithm:

$$y_i = \log(x_i) \text{ for each } x_i$$

- Computations:
- $\log(10) \approx 2.302$
- $\log(20) \approx 2.996$
- $\log(50) \approx 3.912$
- $\log(100) \approx 4.605$
- $\log(200) \approx 5.298$
- $\log(500) \approx 6.215$

The transformed dataset is approximately $([2.302, 2.996, 3.912, 4.605, 5.298, 6.215])$.

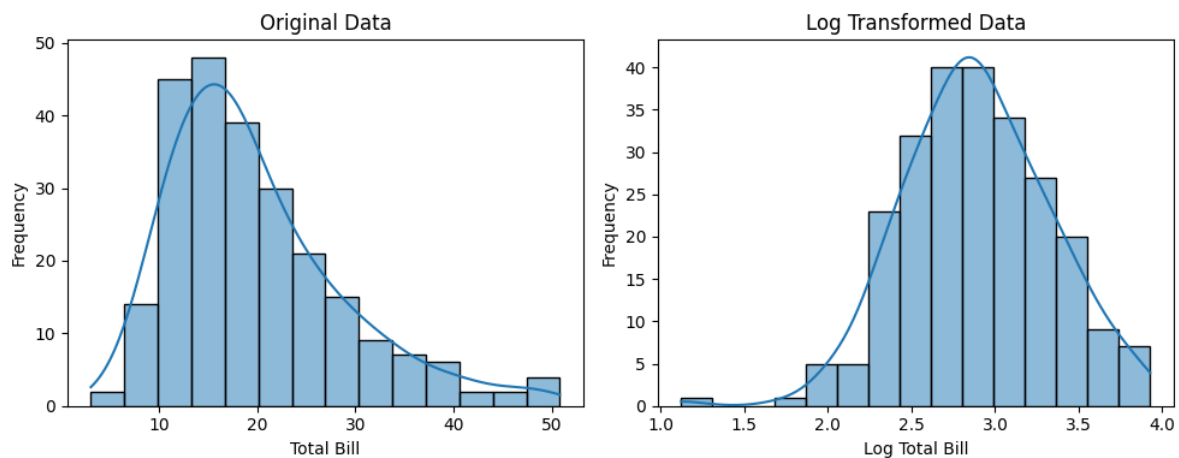


Figure 1: Log Transformation

5.2 Square Root Transformation

Square root transformations are used to stabilize variance and make data more normally distributed, especially with count data.

Mathematical Definition: For a dataset x , the square root transformation is defined as:

$$y = \sqrt{x}$$

Example: Consider a dataset representing counts of a species observed per plot: $([1, 4, 9, 16, 25])$.

1. Transform Data:

- Compute the square root of each element:

$$y_i = \sqrt{x_i}$$

- Computations:

- $\sqrt{1} = 1$
- $\sqrt{4} = 2$
- $\sqrt{9} = 3$
- $\sqrt{16} = 4$
- $\sqrt{25} = 5$

The transformed dataset is ([1, 2, 3, 4, 5]).

5.3 Box-Cox Transformation

The Box-Cox transformation is a family of power transformations that stabilizes variance and makes data more normally distributed. The transformation is given by:

$$y = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \log(x) & \text{if } \lambda = 0 \end{cases}$$

where λ is a transformation parameter that can be estimated from the data.

Example: For a dataset of response times in seconds: ([1, 2, 3, 4, 5]), we can compute Box-Cox for various (λ) values.

- Let's take $\lambda = 0.5$:

$$y = \frac{x^{0.5} - 1}{0.5}$$

- Computations:

- ($\lambda = 0.5$):

- $y_1 = \frac{\sqrt{1}-1}{0.5} = 0$
- $y_2 = \frac{\sqrt{2}-1}{0.5} \approx 0.828$
- $y_3 = \frac{\sqrt{3}-1}{0.5} \approx 1.464$
- $y_4 = \frac{\sqrt{4}-1}{0.5} = 2$
- $y_5 = \frac{\sqrt{5}-1}{0.5} \approx 2.472$

The transformed dataset is approximately ([0, 0.828, 1.464, 2, 2.472]).

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt

# Generate sample data (positive values only, required for Box-Cox)
np.random.seed(42)
data = np.random.exponential(scale=2, size=1000) # Positively skewed data

# Convert to a Pandas DataFrame
df = pd.DataFrame(data, columns=["Original"])

# Apply Box-Cox transformation
df["Transformed"], lambda_value = stats.boxcox(df["Original"])

# Plot before and after transformation
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

ax[0].hist(df["Original"], bins=30, color="skyblue", edgecolor="black")
ax[0].set_title("Original Data (Skewed)")

ax[1].hist(df["Transformed"], bins=30, color="lightcoral", edgecolor="black")
ax[1].set_title(f"Box-Cox Transformed Data (lambda_value: {lambda_value:.2f})")

plt.show()
```

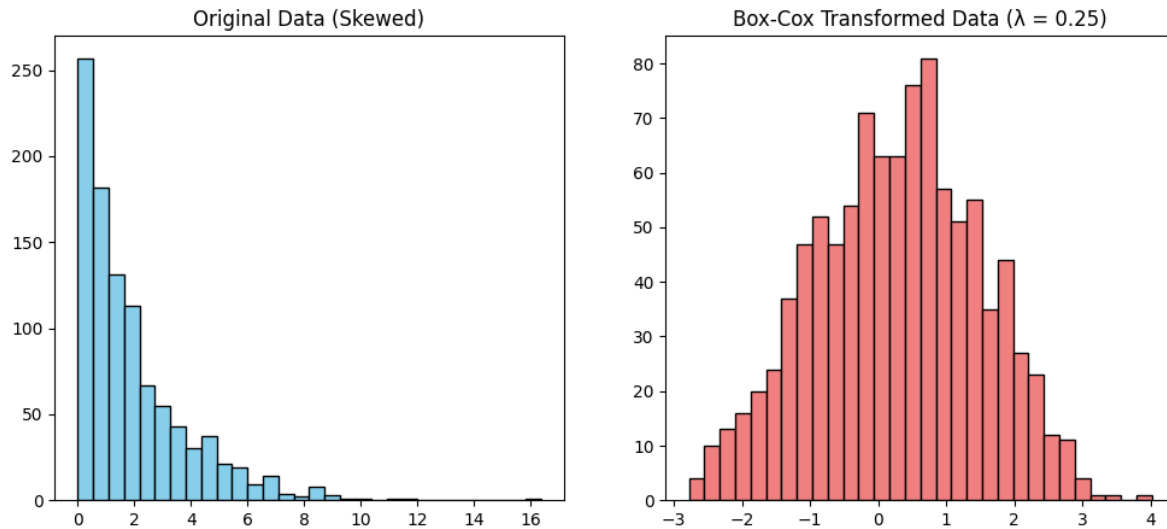


Figure 2: Box-Cox Transformation

5.4 Data Binning

Binning (or discretization) is the process of transforming continuous data into discrete buckets or categories. This can simplify analysis and allow for easier visualization.

1. Equal Width Binning:

- Divide the range of the data into k bin intervals of equal width.

Example: Given a range of test scores: ([55, 88, 61, 48, 73, 85, 92, 78]) and we want to create 4 bins.

- Range of scores: $92 - 48 = 44$
- Width of each bin = $\frac{44}{4} = 11$

Bins will be:

- Bin 1: [48, 59]
- Bin 2: [60, 71]
- Bin 3: [72, 83]
- Bin 4: [84, 95]

Counts in bins:

- Bin 1: 1 score

- Bin 2: 2 scores
- Bin 3: 2 scores
- Bin 4: 3 scores

2. Equal Frequency Binning:

- Each bin contains approximately the same number of observations.

Example: For the same scores sorted:

- Break into 4 equal frequency bins: ([48, 55], [61, 73], [78, 85], [88, 92]).

Counts based on sorted values:

- Bin 1: 2 scores
- Bin 2: 2 scores
- Bin 3: 2 scores
- Bin 4: 2 scores

6 Data Scaling

Scaling helps to transform features to a similar scale.

Many machine learning algorithms are sensitive to the scale of features. For instance, algorithms that rely on distance measures (like k-nearest neighbors and support vector machines) or gradient-based optimization (like linear regression and neural networks) can behave erratically when features are on different scales.

Numerical Example: Consider a dataset containing two features: `height` (in cm) ranging from 150 to 200 and `income` ranging from \$30,000 to \$100,000. If these features are directly used in a distance-based algorithm, the greater range of `income` could overshadow `height`, leading to biased predictions. For example, a distance calculation between two points can be disproportionately influenced by the income feature due to its larger range.

Two of the most commonly used methods for scaling data are **Min-Max Scaling** and **Z-score Standardization**.

6.1 Min-Max Scaling (Normalization)

Min-Max scaling transforms data into a fixed range, usually [0, 1].

The formula for Min-Max scaling is:

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Step-by-step Derivation:

1. Identify the minimum and maximum of the feature.
2. Subtract the minimum from each data point to translate the feature.
3. Divide by the range (max - min) to scale.

Example: Consider the feature age with values: [25, 30, 45, 50].

- The minimum ($X_{\min}$) is 25, and the maximum ($X_{\max}$) is 50.
- For ($X = 30$):

$$X_{\text{scaled}} = \frac{30 - 25}{50 - 25} = \frac{5}{25} = 0.2$$

Repeating this for all values results in scaled ages: [0, 0.2, 0.8, 1].

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt

# Generate sample data
np.random.seed(42)
data = np.random.randint(10, 100, size=(10, 1)) # Random values between 10 and 100

# Convert to DataFrame
df = pd.DataFrame(data, columns=["Original"])

# Apply Min-Max Scaling
scaler = MinMaxScaler() # Scale values between 0 and 1
df["Scaled"] = scaler.fit_transform(df[["Original"]])

# Plot before and after scaling
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

ax[0].hist(df["Original"], bins=5, color="skyblue", edgecolor="black")
```



```
ax[0].set_title("Original Data")

ax[1].hist(df["Scaled"], bins=5, color="lightcoral", edgecolor="black")
ax[1].set_title("Min-Max Scaled Data")

plt.show()
```

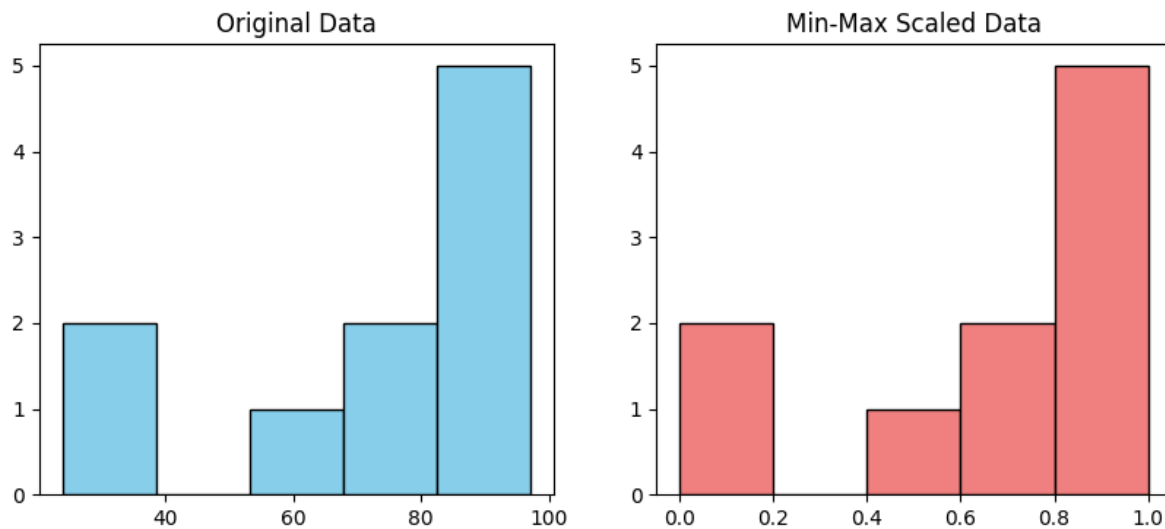


Figure 3: Min-Max Scaling

6.2 Z-score (Standardization)

Z-score standardization transforms data into a distribution with a mean of 0 and a standard deviation of 1.

The formula is:

$$X_{\text{standard}} = \frac{X - \mu}{\sigma}$$

where (μ) is the mean and (σ) is the standard deviation.

Step-by-step Derivation:

1. Compute the mean (μ) of the feature.
2. Compute the standard deviation (σ).
3. For each data point, subtract the mean and divide by the standard deviation.

Numerical Example: Consider the feature `weight` with values: [50, 55, 70, 80].

- Mean (μ):

$$\mu = \frac{50 + 55 + 70 + 80}{4} = 63.75$$

- Standard deviation (σ):

$$\sigma = \sqrt{\frac{(50 - 63.75)^2 + (55 - 63.75)^2 + (70 - 63.75)^2 + (80 - 63.75)^2}{4}} = 12.5$$

- For ($X = 70$):

$$X_{\text{standard}} = \frac{70 - 63.75}{12.5} = 0.5$$

The standardized weights will be: [-1.1, -0.7, 0.5, 1.3].

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Generate sample data (real numbers)
np.random.seed(42)
data = np.random.uniform(50.0, 150.0, size=(10, 1)) # Random real numbers between 50.0 and 150.0

# Convert to DataFrame
df = pd.DataFrame(data, columns=["Original"])

# Apply Z-score Standardization
scaler = StandardScaler()
df["Standardized"] = scaler.fit_transform(df[["Original"]])

# Print original and standardized values
print(df)

# Plot before and after standardization
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

ax[0].hist(df["Original"], bins=5, color="skyblue", edgecolor="black")
ax[0].set_title("Original Data")

ax[1].hist(df["Standardized"], bins=5, color="lightcoral", edgecolor="black")
```

```
ax[1].set_title("Z-Score Standardized Data")  
  
plt.show()
```

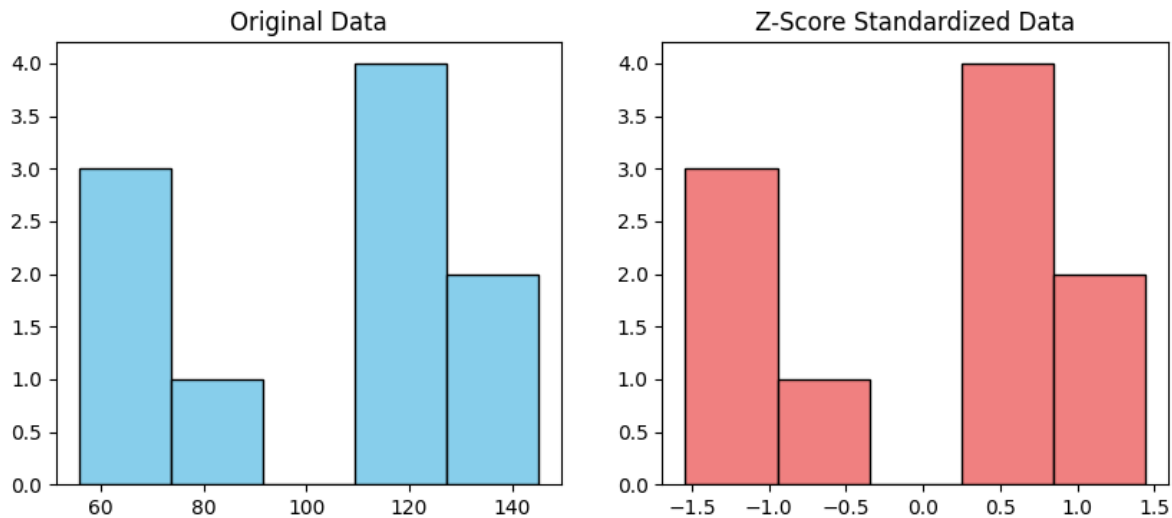


Figure 4: Z-score Standardization

6.3 Differences between Normalization and Standardization

Normalization and standardization are both techniques for adjusting the scale of data, but they serve different purposes and result in different output:

- **Normalization** (Min-Max Scaling):
 - Compresses the data into a specific range [0, 1].
 - Useful for algorithms that require bounded input (e.g., neural networks).
- **Standardization** (Z-score Normalization):
 - Centers the data by subtracting the mean and dividing by the standard deviation:

7 Variable Selection

Variable selection, also known as feature selection, is a crucial step in building a predictive model. It involves eliminating irrelevant or redundant variables to improve model performance and interpretability.

7.1 Forward Elimination

Forward selection is a stepwise regression approach where we start with no variables in the model. At each step, we add the variable that improves the model fit the most until no further significant improvement can be made.

Steps:

1. Start with an empty model.
2. For each predictor not in the model, fit the model and calculate a performance metric (e.g., AIC, BIC).
3. Choose the variable that improves the metric the most.
4. Repeat steps 2 and 3 until no variables improve the model significantly.

Example: Suppose we have predictors (X_1) , (X_2) , and (X_3) . We find:

- After fitting (X_1) alone, $AIC = 200$.
- Fitting (X_2) gives $AIC = 195$.
- Fitting (X_3) gives $AIC = 198$.

Choose (X_2) and repeat the process.

7.2 Backward Elimination

Backward elimination starts with all candidate variables in the model and systematically removes the least significant variables.

Steps:

1. Start with all predictors in the model.
2. At each step, remove the variable with the highest p-value.
3. Continue the process until all remaining variables have p-values below a pre-determined threshold (commonly 0.05).

Example: Consider the model with predictors (X_1) , (X_2) , (X_3) where their p-values are:

- $p(X_1) = 0.01$
- $p(X_2) = 0.06$
- $p(X_3) = 0.03$

Remove (X_2) and repeat until all p-values are significant.

7.3 Importance of Variable Selection in Modeling

Effective variable selection has numerous benefits in model development:

1. **Improved Model Interpretability:** Fewer predictors make the model easier to understand and communicate.
2. **Enhanced Predictive Performance:** Removing irrelevant variables reduces noise and leads to better generalization on unseen data.
3. **Reduced Computational Cost:** Fewer features decrease the time required for model fitting and validation.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.metrics import mean_squared_error

# Generate Synthetic Data
np.random.seed(42)
X = np.random.rand(100, 5) # 100 samples, 5 features
y = 3 * X[:, 0] + 2 * X[:, 1] - 1.5 * X[:, 2] + np.random.randn(100)
# Target variable with noise

df = pd.DataFrame(X, columns=[f'Feature_{i}' for i in range(1, 6)])
df['Target'] = y

# Split Data into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(df.drop(columns=["Target"]), \
    df["Target"], test_size=0.2, random_state=42)

# Initialize Model
model = LinearRegression()

# Forward Feature Selection (Adding Features Step by Step)
forward_selector = SequentialFeatureSelector(model, n_features_to_select="auto", \
```

```

    direction="forward", cv=5)
forward_selector.fit(X_train, y_train)
selected_forward = X_train.columns[forward_selector.get_support()]
print(f"Selected Features (Forward): {list(selected_forward)}")

# Backward Feature Selection (Removing Features Step by Step)
backward_selector = SequentialFeatureSelector(model, n_features_to_select="auto", \
    direction="backward", cv=5)
backward_selector.fit(X_train, y_train)
selected_backward = X_train.columns[backward_selector.get_support()]
print(f"Selected Features (Backward): {list(selected_backward)}")

# Train and Evaluate Models
X_train_forward, X_test_forward = X_train[selected_forward], \
    X_test[selected_forward]
X_train_backward, X_test_backward = X_train[selected_backward], \
    X_test[selected_backward]

# Train forward selection model
model.fit(X_train_forward, y_train)
y_pred_forward = model.predict(X_test_forward)
mse_forward = mean_squared_error(y_test, y_pred_forward)

# Train backward selection model
model.fit(X_train_backward, y_train)
y_pred_backward = model.predict(X_test_backward)
mse_backward = mean_squared_error(y_test, y_pred_backward)

print(f"Forward Selection MSE: {mse_forward:.4f}")
print(f"Backward Selection MSE: {mse_backward:.4f}")

```

```

Selected Features (Forward): ['Feature_1', 'Feature_2']
Selected Features (Backward): ['Feature_1', 'Feature_2', 'Feature_3']
Forward Selection MSE: 1.1239
Backward Selection MSE: 0.9299

```

8 Feature Engineering

It involves the creation and transformation of input variables from raw data to improve model performance.

8.1 Polynomial Features

Polynomial features are used to capture non-linear relationships between the input variables. By transforming the original features into polynomial combinations, you can allow for more flexibility in the modeling process.

Derivation

For a feature vector $X = [x_1, x_2, \dots, x_n]$, the polynomial features for degree 2 can be generated as follows:

$$\Phi(X) = [1, x_1, x_2, x_1^2, x_1x_2, x_2^2]$$

8.2 Interaction Terms

Interaction terms quantify how the combination of two or more features influences the target variable. They become particularly useful when the effect of one feature on the target variable is dependent on the value of another feature.

Derivation

For features x_1 and x_2 :

$$\text{InteractionTerm} = x_1 \cdot x_2$$

Numerical Examples

1. Effect of Temperature and Time:

- If a study considers temperature (x_1) and time of day (x_2), the interaction x_1x_2 might better predict energy consumption during peak usage times.

2. Advertising and Seasonality:

- For a retailer, the effect of advertising spend might vary seasonally. If x_1 is advertising spend and x_2 is a season variable (1 for holiday, 0 otherwise), $x_1 \times x_2$ captures this seasonal interaction.

3. Health Outcomes:

- In a healthcare dataset, age (x_1) and existing conditions (x_2) could interact where the elderly with certain conditions have higher health risks.

4. Insurance Premiums:

- For insurance models, combining age of a driver (x_1) and vehicle speed (x_2) as an interaction term can reveal insights about risk that linear terms alone cannot.

9 Underfitting and Overfitting

9.1 Definition

Underfitting and overfitting are two common problems in training a model that can significantly affect the performance of predictive models.

Underfitting occurs when a model is too simple to capture the underlying patterns in the data. It fails to learn the relevant features, leading to poor predictive performance on both the training and testing datasets.

Overfitting, on the other hand, occurs when a model captures noise along with the underlying data pattern. This results in excellent performance on the training data but poor generalization to unseen data.

The consequences of underfitting and overfitting can be severe:

- **Underfitting** leads to high bias, resulting in low accuracy on both training and testing data. For example, a linear regression model trying to fit a quadratic dataset will miss the curves, leading to significant prediction errors.
- **Overfitting** results in high variance. The model may perform exceptionally well on training data, but its performance on testing data will be erratic, as it becomes sensitive to noise. For instance, a polynomial regression model of high degree will create a complex curve that fits the training data perfectly but fails to predict new data points accurately.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

# Generate Synthetic Data
np.random.seed(42)
```



```

X = np.linspace(-3, 3, 50).reshape(-1, 1)
y = X**3 - 3*X + np.random.randn(50, 1) * 3

# Split Data into Train & Test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train and Plot Models with Different Degrees
plt.figure(figsize=(12, 5))
degrees = [1, 3, 10] # Linear, Cubic, Overfitting
titles = ["Underfitting (Linear)", "Good Fit (Cubic)", "Overfitting (Degree=10)"]

for i, degree in enumerate(degrees):
    poly = PolynomialFeatures(degree=degree)
    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    model = LinearRegression().fit(X_train_poly, y_train)
    y_pred = model.predict(X_test_poly)
    mse = mean_squared_error(y_test, y_pred)

    # Plot Results
    plt.subplot(1, 3, i+1)
    plt.scatter(X_train, y_train, color="blue", label="Train Data")
    plt.scatter(X_test, y_test, color="red", label="Test Data")

    X_range = np.linspace(-3, 3, 100).reshape(-1, 1)
    plt.plot(X_range, model.predict(poly.transform(X_range)), color="black", linewidth=2)

    plt.title(f"{titles[i]}\nMSE: {mse:.2f}")
    plt.legend()

plt.tight_layout()
plt.show()

```

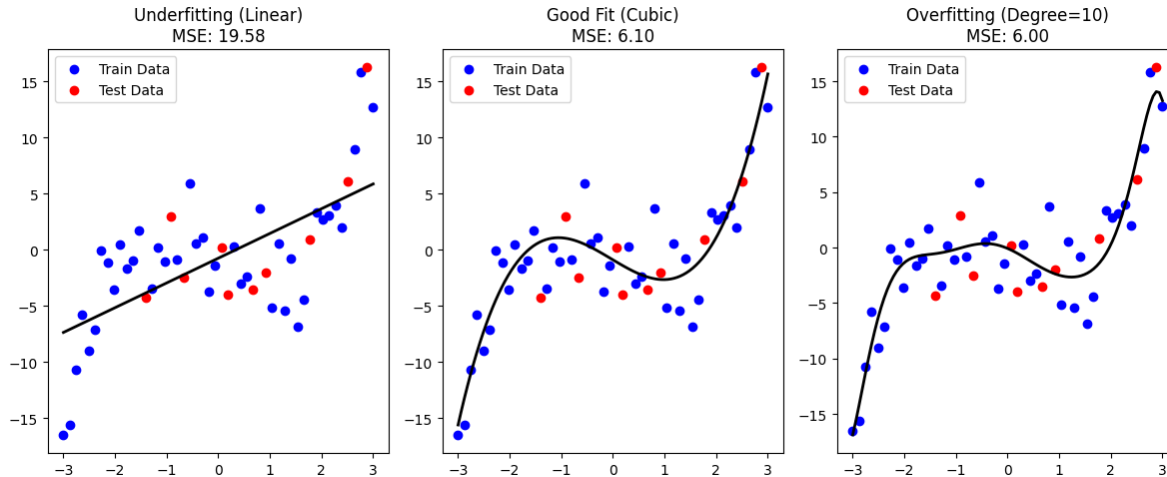


Figure 5: Illustration of underfitting, good fit and overfitting

9.2 Ridge Regression (L2 Regularization)

Regularization techniques are key to preventing overfitting by adding a penalty for model complexity. Two popular methods are Lasso and Ridge regression:

Ridge regression is a linear regression technique that reduces overfitting by adding a penalty to large coefficients.

$$\min_{\beta} \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right)$$

where (λ) is the regularization parameter.

- **First term**

$$\sum (y_i - \hat{y}_i)^2$$

is the usual squared error (minimizing prediction error).

- **Second term**

$$\lambda \sum \beta_j^2$$

is the penalty term that discourages large coefficient values.

λ (Regularization parameter) controls the strength of regularization. $\lambda = 0$ Regular linear regression. Higher λ means more shrinkage (smaller coefficients).

9.3 Lasso Regression (L1 Regularization)

Lasso (Least Absolute Shrinkage and Selection Operator) is a linear regression technique that adds a penalty based on the **absolute values** of the coefficients. This helps in feature selection by shrinking some coefficients to **zero**.

$$\min_{\beta} \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j| \right)$$

where:

- **First term**

$$\sum (y_i - \hat{y}_i)^2$$

minimizes the squared error (like ordinary least squares).

- **Second term**

$$\lambda \sum |\beta_j|$$

is the L1 penalty that forces some coefficients to shrink to zero.

$\lambda = 0$ Regular linear regression (no penalty). Higher λ implies more coefficients shrink to zero (stronger feature selection).

Lasso can effectively reduce irrelevant features, leading to a simpler and more interpretable model. Lasso can also be used more variable selection.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso, Ridge
from sklearn.metrics import mean_squared_error

# Generate Synthetic Data
np.random.seed(42)
X = 2 * np.random.rand(100, 1) # 100 samples, 1 feature
y = 4 + 3 * X[:, 0] + np.random.randn(100) # y = 4 + 3X + noise

# Split Data into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Lasso Regression Model
lasso_model = Lasso(alpha=0.1) # Alpha is the regularization parameter
lasso_model.fit(X_train, y_train)
```

```
# Train Ridge Regression Model
ridge_model = Ridge(alpha=0.1)
ridge_model.fit(X_train, y_train)

# Make Predictions
y_pred_lasso = lasso_model.predict(X_test)
y_pred_ridge = ridge_model.predict(X_test)

# Evaluate Models (MSE)
lasso_mse = mean_squared_error(y_test, y_pred_lasso)
ridge_mse = mean_squared_error(y_test, y_pred_ridge)

print(f"Lasso Regression MSE: {lasso_mse:.4f}")
print(f"Ridge Regression MSE: {ridge_mse:.4f}")

# Visualizing the Results
plt.scatter(X_test, y_test, color="blue", label="Actual Data")
plt.plot(X_test, y_pred_lasso, color="red", label="Lasso Regression")
plt.plot(X_test, y_pred_ridge, color="green", label="Ridge Regression")
plt.legend()
plt.xlabel("Feature X")
plt.ylabel("Target y")
plt.title("Lasso vs Ridge Regression")
plt.show()
```

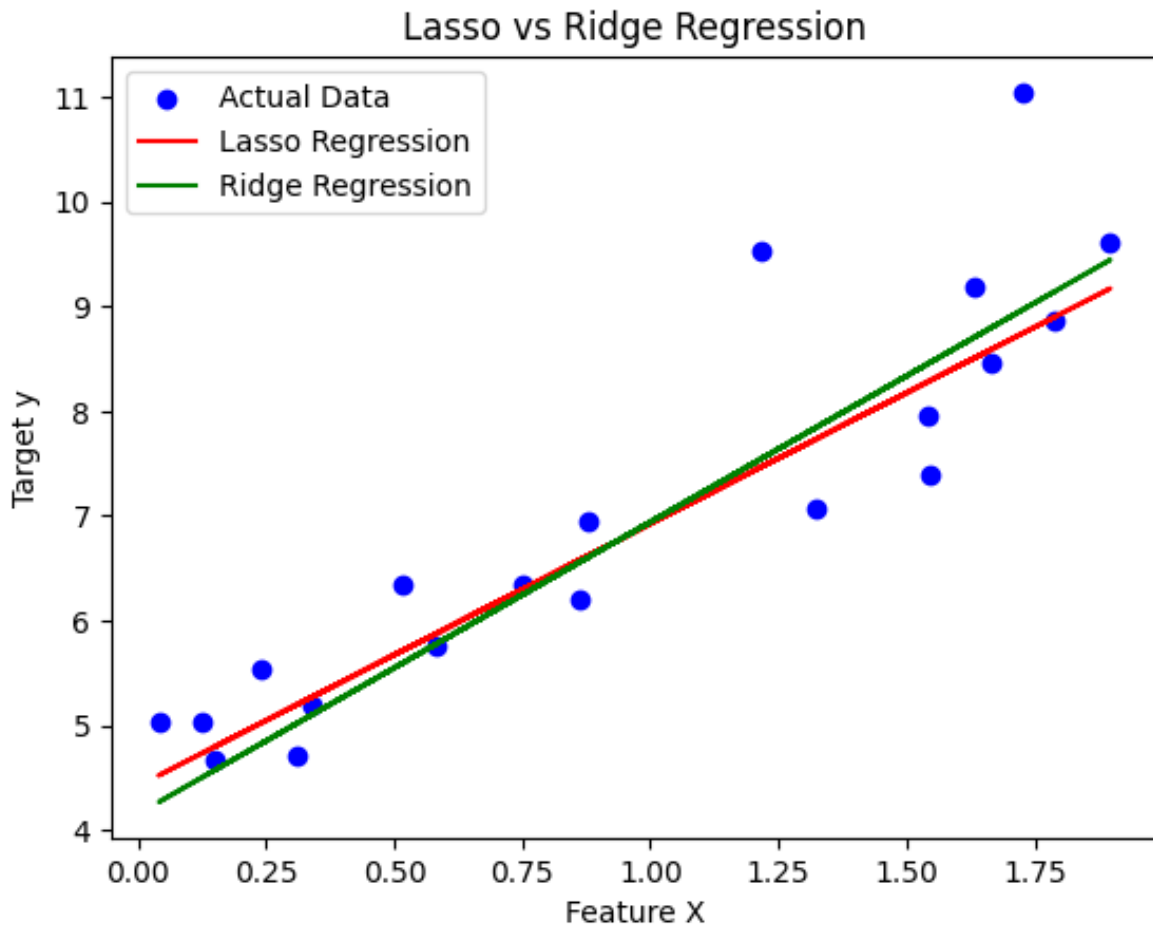


Figure 6: Lasso and Ridge Regression

10 Model Evaluation

Model evaluation is a crucial step in the predictive analytics as it determines how well a model performs on unseen data. Proper evaluation helps to ensure that the model is robust, generalizable, and suitable for deployment.

10.1 Accuracy

Accuracy is the ratio of correctly predicted instances to the total instances.

Actual \ Predicted	Positive (1)	Negative (0)
Positive (1)	True Positive (TP)	False Negative (FN)
Negative (0)	False Positive (FP)	True Negative (TN)

- **True Positive (TP):** Model correctly predicts positive (e.g., detects disease when it's present).
- **False Negative (FN):** Model predicts negative, but it's actually positive (missed detection).
- **False Positive (FP):** Model predicts positive, but it's actually negative (false alarm).
- **True Negative (TN):** Model correctly predicts negative (no false alarm).

Accuracy is expressed mathematically as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- (TP) = True Positive
- (TN) = True Negative
- (FP) = False Positive
- (FN) = False Negative

Example: Suppose we have a binary classifier that predicts whether a patient has a disease. Out of 100 patients, 70 are correctly identified as healthy (TN), 20 as having the disease (TP), while 5 are incorrectly classified as healthy (FP), and 5 as having the disease (FN).

Consider a confusion matrix from this binary classification task:

Actual \ Predicted	Positive (P)	Negative (N)
Positive (P)	20 (TP)	5 (FN)
Negative (N)	5 (FP)	70 (TN)

$$\text{Accuracy} = \frac{20 + 70}{100} = 0.9 \text{ or } 90\%$$

10.2 Precision

Precision measures the accuracy of positive predictions and is given by:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Example: Continuing with the previous example, the precision would be calculated as:

$$\text{Precision} = \frac{20}{20 + 5} = \frac{20}{25} = 0.8 \text{ or } 80\%$$

10.3 Recall

Recall, also known as Sensitivity or True Positive Rate, measures the model's ability to find all relevant instances:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Example: In our scenario:

$$\text{Recall} = \frac{20}{20 + 5} = \frac{20}{25} = 0.8 \text{ or } 80\%$$

10.4 F1-Score

The F1-Score is the harmonic mean of precision and recall, providing a single score that balances the two metrics:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Example: Using the previous precision and recall values:

$$F1 = 2 \times \frac{0.8 \times 0.8}{0.8 + 0.8} = 2 \times \frac{0.64}{1.6} = 0.8$$

10.5 ROC-AUC

Receiver Operating Characteristic (ROC) curve plots the true positive rate against the false positive rate at various threshold levels. The Area Under the Curve (AUC) quantifies the overall ability of the model to discriminate between positive and negative classes.

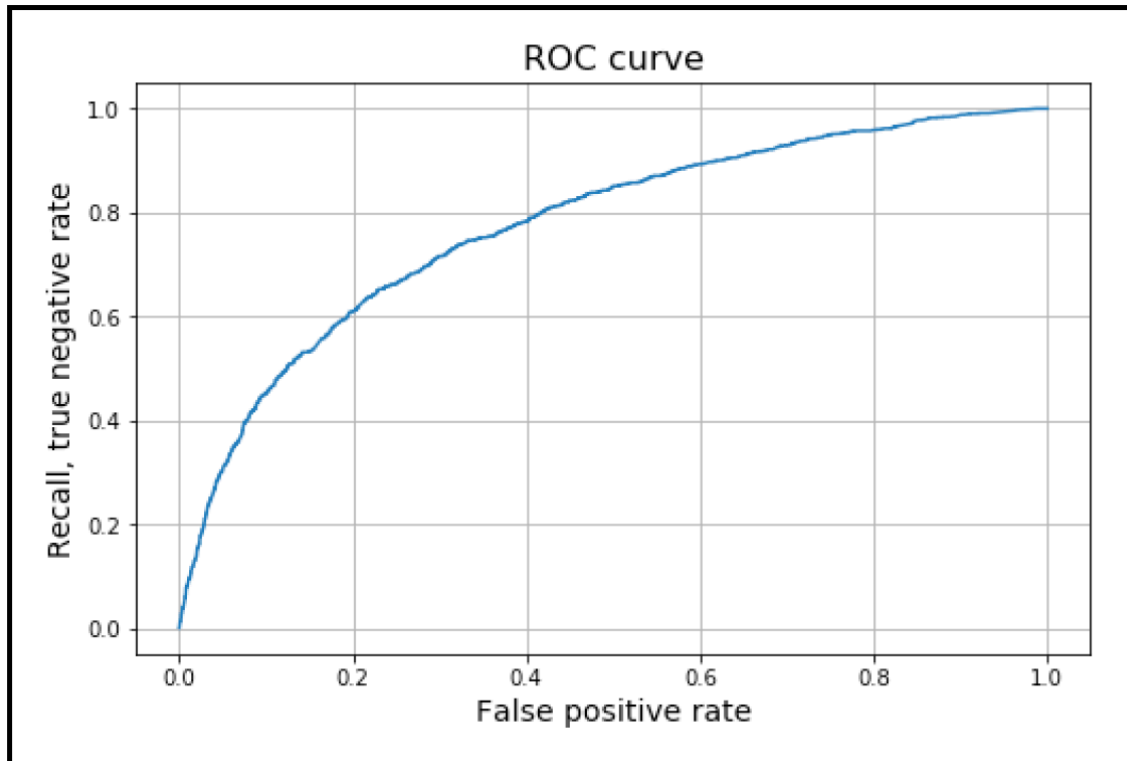


Figure 7: ROC Plot

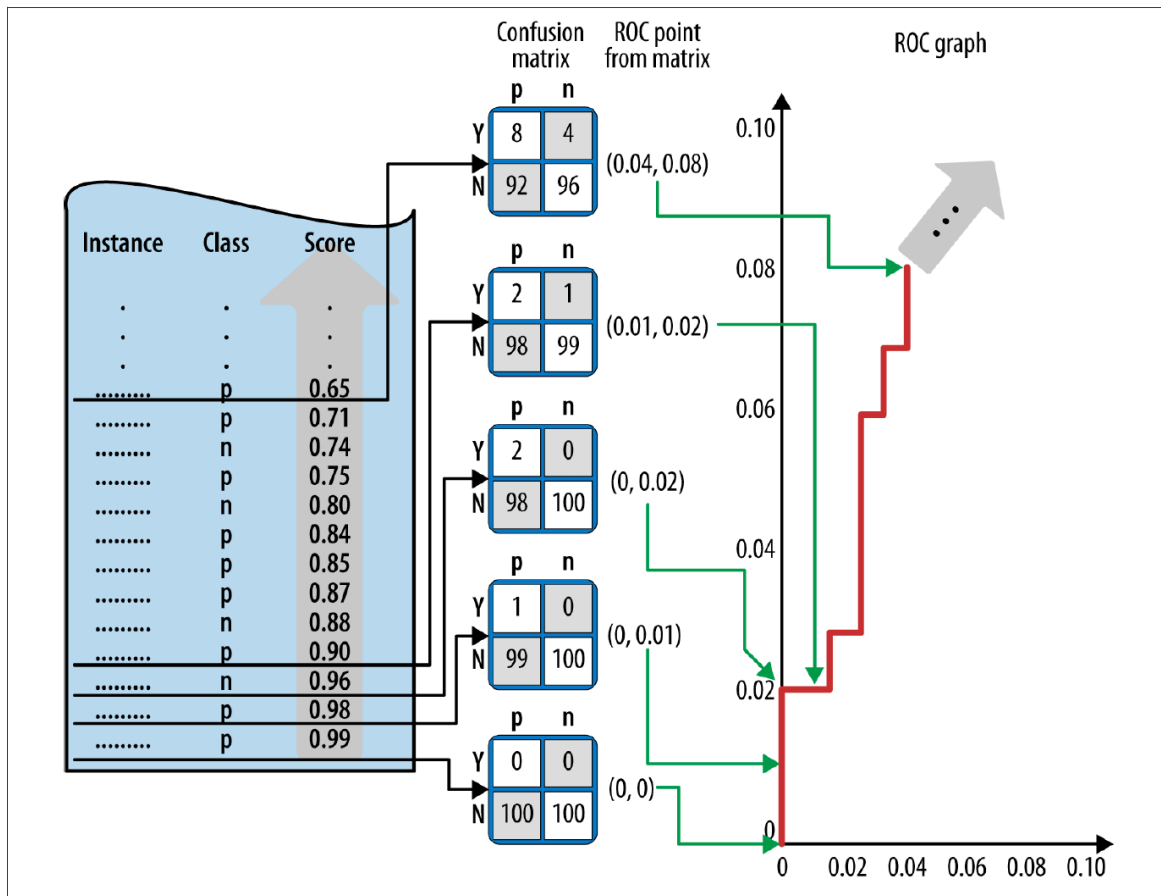


Figure 8: How ROC plots are constructed

Example: If we have a model that achieves an AUC of 0.85, it indicates that the model has an 85% chance of correctly distinguishing between classes.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, f1_score, roc_auc_score, roc_curve
import matplotlib.pyplot as plt

# Generate Synthetic Classification Data
X, y = make_classification(n_samples=1000, n_features=20, random_state=42)

# Split Data into Training and Testing Sets
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Logistic Regression Model
model = LogisticRegression()
model.fit(X_train, y_train)

# Make Predictions
y_pred = model.predict(X_test)
y_pred_prob = model.predict_proba(X_test)[:, 1] # Probabilities for AUC

# Calculate Precision, Recall, F-score, AUC
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f_score = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_pred_prob)

# Calculate Sensitivity and Specificity
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()

sensitivity = tp / (tp + fn) # Also called True Positive Rate or Recall
specificity = tn / (tn + fp) # True Negative Rate

# Print the results
print(f"Precision: {precision:.2f}")
print(f"Recall (Sensitivity): {recall:.2f}")
print(f"F-score: {f_score:.2f}")
print(f"AUC: {auc:.2f}")
print(f"Sensitivity: {sensitivity:.2f}")
print(f"Specificity: {specificity:.2f}")

# Plot ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)

plt.figure(figsize=(6, 6))
plt.plot(fpr, tpr, color='blue', label=f'AUC = {auc:.2f}')
plt.plot([0, 1], [0, 1], color='gray', linestyle='--') # Diagonal line (random classifier)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()

```

Precision: 0.91
Recall (Sensitivity): 0.80
F-score: 0.86
AUC: 0.92
Sensitivity: 0.80
Specificity: 0.91

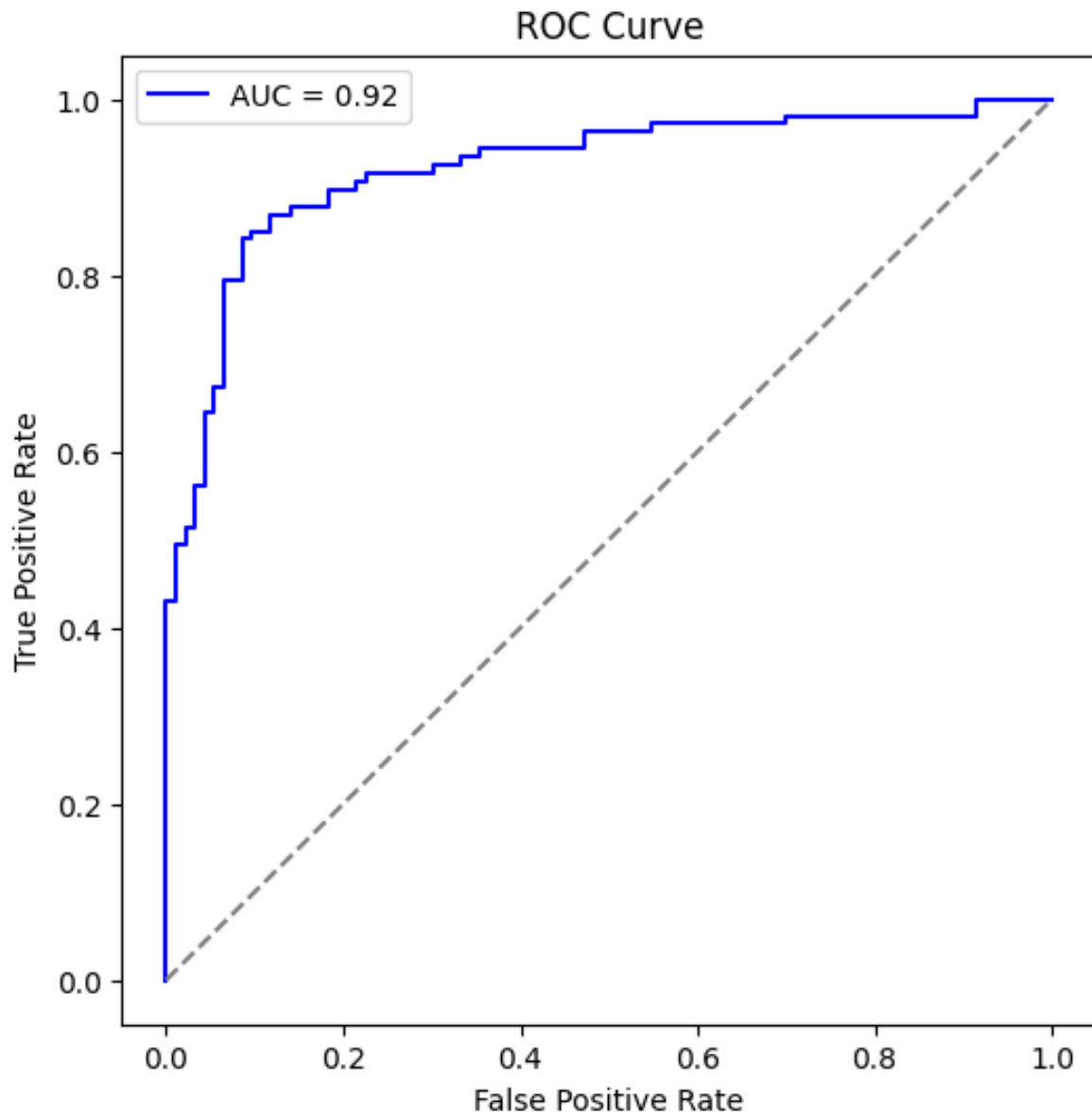


Figure 9: Receiver operating characteristic curve

10.6 K-Fold Cross-Validation

Cross-validation is a statistical method used to assess the performance of a model. The main purpose is to ensure that the evaluation of the model is reliable and generalizable.

In K-fold cross-validation, the data set is divided into K equally sized folds. The model is trained on K-1 folds and tested on the remaining one. This process is repeated K times, each time using a different fold as the testing set.

Steps:

- Split the Dataset
 - The dataset is divided into K equal parts (folds).
 - Each fold will take a turn being the test set, while the remaining folds are used for training.
- Train and Validate for Each Fold
 - The model is trained on all folds except one.
 - The left-out fold is used as a test set to check the model's performance.
- This process repeats **K times**, so every fold gets tested once.
- After testing on all folds, the results (e.g., accuracy, precision, recall) are **averaged**.

Example: For a dataset of 100 samples and K=5, each fold will consist of 20 samples. This method not only utilizes all available data for training but also provides a robust estimate of model performance.

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_regression
from sklearn.model_selection import KFold, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Generate a Synthetic Regression Dataset
X, y = make_regression(n_samples=1000, n_features=20, noise=10, random_state=42)
```

```
# Set Up K-Fold Cross-Validation
k = 5 # Number of folds
kf = KFold(n_splits=k, shuffle=True, random_state=42)

# Initialize Model
model = LinearRegression()

# Perform Cross-Validation and Get Scores (Using Negative MSE)
mse_scores = cross_val_score(model, X, y, cv=kf, scoring='neg_mean_squared_error')

# Convert Negative MSE to Positive Values
mse_scores = -mse_scores # Since cross_val_score returns negative MSE

# Print Results
print(f"MSE for each fold: {mse_scores}")
print(f"Mean MSE: {np.mean(mse_scores):.4f}")
```

```
MSE for each fold: [114.27751339  94.62469548 105.45154013 114.41539632 106.57179726]
Mean MSE: 107.0682
```

11 Bias and Variance

In machine learning, the bias-variance trade-off is a essential concept that describes the balance between two sources of error that affect the performance of a predictive model: bias and variance.

- **Bias** refers to the error introduced by approximating a real-world problem, which may be complex, with a simplified model. High bias can cause an algorithm to miss the relevant relations between features and target outputs (underfitting).
- **Variance** refers to the error introduced by the model's sensitivity to fluctuations in the training dataset. High variance can cause the model to model the random noise in the training data rather than the intended outputs (overfitting).